

Project Source Code Implementation

Complete modular code implementation for the Enterprise Generative AI project [cite: 1, 2].

1. Configuration Module (`config.py`)

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    app_name: str = "Enterprise GenAI Service"
    model_name: str = "meta-llama/Llama-3-8b-chat-hf"
    embedding_model: str = "sentence-transformers/all-MiniLM-L6-v2"
    chroma_persist_directory: str = "./chroma_db"
    max_tokens: int = 1024
    temperature: float = 0.1

    class Config:
        env_file = ".env"

settings = Settings()
```

2. RAG Engine & Vector Store (`rag\_engine.py`)

```
import os
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma

class EnterpriseRAG:
    def __init__(self, persist_dir: str):
        self.embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
        self.vector_store = Chroma(persist_directory=persist_dir, embedding_function=self.embeddings)
        self.retriever = self.vector_store.as_retriever(search_kwargs={"k": 3})

    def add_documents(self, texts: list[str], metadatas: list[dict] = None):
        splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
        docs = splitter.create_documents(texts, metadatas=metadatas)
        self.vector_store.add_documents(docs)
        self.vector_store.persist()

    def query(self, question: str) -> str:
        relevant_docs = self.retriever.get_relevant_documents(question)
        context = "\n".join([doc.page_content for doc in relevant_docs])
        return f"Generated Answer based on context: {context[:200]}..."
```

3. FastAPI Main Application (`main.py`)

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from rag_engine import EnterpriseRAG
from config import settings

app = FastAPI(title=settings.app_name, version="1.0.0")
rag = EnterpriseRAG(persist_dir=settings.chroma_persist_directory)

class QueryRequest(BaseModel):
    prompt: str

class DocumentRequest(BaseModel):
    texts: list[str]

@app.post("/api/v1/query")
def run_query(request: QueryRequest):
    try:
        answer = rag.query(request.prompt)
        return {"status": "success", "response": answer}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/api/v1/ingest")
def ingest_documents(request: DocumentRequest):
    try:
        rag.add_documents(request.texts)
        return {"status": "success", "message": f"Ingested {len(request.texts)} documents."}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

4. Containerization (`Dockerfile`)

```
FROM python:3.10-slim AS builder

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

FROM python:3.10-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY . .

ENV PATH=/root/.local/bin:$PATH
EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

5. Dependency Manifest (`requirements.txt`)

```
fastapi==0.110.0
uvicorn==0.28.0
pydantic==2.6.4
pydantic-settings==2.2.1
langchain==0.1.12
langchain-community==0.0.28
chromadb==0.4.24
sentence-transformers==2.5.1
torch==2.2.1
weasyprint==60.2
```